

АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ (ROS 2)

Робот-курьер НТЦ АО «АВТОВАЗ», шасси 4WIS/4WID с крабовым ходом. Целевая платформа: **Ubuntu 24.04 / 26.04 LTS, ROS 2 Humble/Jazzy**.

Документ описывает полную программную архитектуру: пакеты, узлы, топики, сервисы, действия, дерево TF, протокол обмена с микроконтроллерами и порядок запуска. Интерфейсы оператора (GUI) в данной ревизии **не реализуются** — весь функционал обеспечивается узлами и конфигурацией.

1. Уровни управления

УРОВЕНЬ 4. МИССИЯ И БЕЗОПАСНОСТЬ

rus_slam_safety: mission_manager_node (FSM миссии, доставка по точкам),
safety_node (гейт скоростей, e-stop, watchdog, АКБ)

УРОВЕНЬ 3. НАВИГАЦИЯ И ВОСПРИЯТИЕ

Nav2 (планирование, recovery), slam_toolbox (SLAM), AMCL (локализация),
rus_slam_perception: camera_node, sign_detector_node, traffic_light_node
rus_slam_lidar: lds01_driver_node (Лидар LDC 01 -> LaserScan)

УРОВЕНЬ 2. УПРАВЛЕНИЕ ДВИЖЕНИЕМ

rus_slam_base: crab_drive_node (обратная кинематика 4WIS/4WID),
odometry_node (одометрия + TF + JointState)

УРОВЕНЬ 1. МОСТ ОБЩЕНИЯ С ЖЕЛЕЗОМ

rus_slam_base: module_bridge_node (4× UART 115200, CRC16, симуляция)

УРОВЕНЬ 0. МИКРОКОНТРОЛЛЕРЫ (4×)

firmware/module_controller: Arduino Mega Pro + TB6600 + свой BLDC-драйвер

2. Состав пакетов (колкон-воркспейс ros2_ws/src)

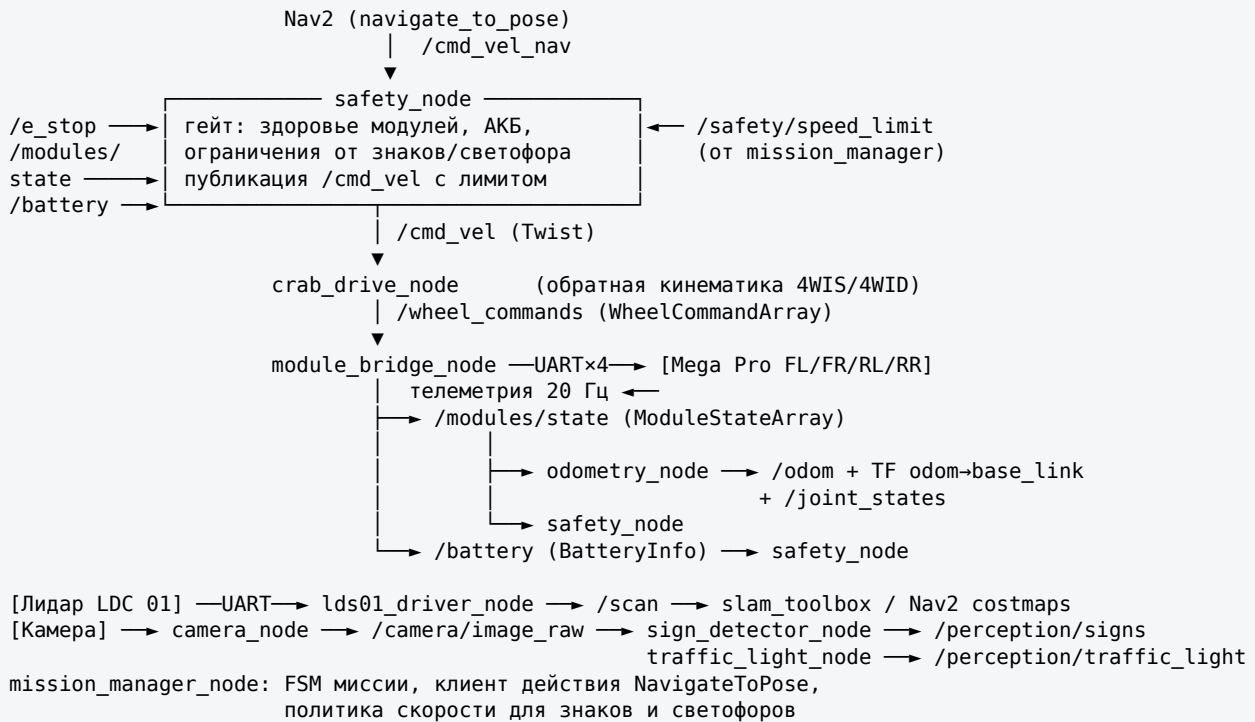
Пакет	Тип	Назначение
rus_slam_interfaces	ament_cmake (rosidl)	Пользовательские сообщения/сервисы: команды колес, состояние модулей, АКБ, знаки, светофор, хоминг
rus_slam_base	ament_python	Ядро: протокол обмена, кинематика, мост UART, одометрия, TF
rus_slam_description	ament_python	URDF/Xacro модель робота + robot_state_publisher
rus_slam_lidar	ament_python	Драйвер Лидара LDC 01 (UART -> sensor_msgs/LaserScan)
rus_slam_perception	ament_python	Камера, детекция знаков («Стоп», «Пешеходный переход», «Искусственная неровность») и светофоров
rus_slam_navigation	ament_python	Конфиги и лаунчеры: slam_toolbox, Nav2, AMCL
rus_slam_safety	ament_python	Безопасность (гейт /cmd_vel, watchdog, АКБ) и менеджер миссии (FSM)
rus_slam_bringup	ament_python	Точка входа: все лаунч-файлы, конфигурация, udev-правила

Зависимости сборки: `colcon build --symlink-install` в каталоге `ros2_ws`. Внешние зависимости: `python3-serial`, `python3-opencv`, `slam_toolbox`, `nav2_bringup`, `robot_state_publisher`, `хасро`. Установка одним списком:

```
sudo apt install -y python3-serial python3-opencv \  
ros-$ROS_DISTRO-slam-toolbox ros-$ROS_DISTRO-navigation2 \
```

```
ros-$ROS_DISTRO-nav2-bringup ros-$ROS_DISTRO-robot-state-publisher \
ros-$ROS_DISTRO-xacro ros-$ROS_DISTRO-tf-transformations
```

3. Граф узлов и топиков



3.1. Таблица топиков

Топик	Тип	И з д а т е л ь	Подписчики	Частота
/cmd_vel_nav	geometry_msgs/Twist	Nav2	safety_node	до 20 Гц
/cmd_vel	geometry_msgs/Twist	safety_node	crab_drive_node	20 Гц
/wheel_commands	rus_slam_interfaces/WheelCommandArray	crab_drive_node	module_bridge_node	до 50 Гц
/modules/state	rus_slam_interfaces/ModuleStateArray	module_bridge_node	odometry_node, safety_node	20 Гц
/battery	rus_slam_interfaces/BatteryInfo	module_bridge_node	safety_node	1 Гц
/odom	nav_msgs/Odometry	odometry_node	Nav2, slam_toolbox	20 Гц
/joint_states	sensor_msgs/JointState	odometry_node	robot_state_publisher	20 Гц

Топик	Тип	И з д а т е л ь	Подписчики	Частота
/scan	sensor_msgs/LaserScan	ld s0 1_ dri ve r_ no de	SLAM, costmaps	5-10 Гц
/camera/image_raw	sensor_msgs/Image (rgb8)	ca m er a_ no de	детекторы	15-30 Гц
/camera/camera_info	sensor_msgs/CameraInfo	ca m er a_ no de	—	15-30 Гц
/perception/signs	rus_slam_interfaces/SignDetectionArray	si gn _d et ec to r_ no de	mission_manager	5-15 Гц
/perception/traffic_light	rus_slam_interfaces/TrafficLightState	tr aff ic_ lig ht_ n od e	mission_manager	5-15 Гц
/e_stop	std_msgs/Bool	вн е ш ни й ис то чн ик (к но пк а/ пу ль т)	safety_node	событийно

Топик	Тип	И з д а т е л ь	Подписчики	Частота
/safety/status	std_msgs/String	sa fe ty _n od e	mission_manager	2 Гц
/safety/speed_limit	std_msgs/Float32	mi ssi on _m an ag er	safety_node	событийно
/tf, /tf_static	tf2_msgs/TFMessage	вс е	все	—

3.2. Сервисы и действия

Имя	Тип	Поставщик	Назначение
/home_modules	rus_slam_interfaces/HomeModules	module_bridge_node	Поиск нулевого азимута (концевики)
/modules/enable	rus_slam_interfaces/SetModulesEnabled	module_bridge_node	Включение/снятие момента с модулей
/navigate_to_pose	nav2_msgs/action/NavigateToPose	Nav2	Целевая точка маршрута (клиент — mission_manager)

3.3. Дерево TF

```
map —(Nav2/slam_toolbox/AMCL)→ odom —(odometry_node)→ base_link
├─ base_footprint (static)
├─ lidar_link (static)
├─ camera_link (static)
├─ module_fl_steel → wheel_fl (continuous)
├─ module_fr_steel → wheel_fr
├─ module_rl_steel → wheel_rl
├─ module_rr_steel → wheel_rr
```

Параметры map→odom дает навигационный стек; odom→base_link — узел одометрии по телеметрии энкодеров; поворотные рамы и колеса анимируются через /joint_states.

4. Ключевые алгоритмы

4.1. Обратная кинематика (rus_slam_base/kinematics.py)

Вход: (vx, vy, wz). Для каждого модуля i с координатами (x_i, y_i):

```
vx_i = vx - wz·y_i; vy_i = vy + wz·x_i
v_i = hypot(vx_i, vy_i); θ_i = atan2(vy_i, vx_i)
```

Оптимизация перекладки: если $|\theta_i - \theta_{prev}| > 90^\circ$, угол разворачивается на 180° , скорость инвертируется (колесо едет «задом» — быстрее, чем доворот). Нормализация: если максимум v_i превышает max_linear_speed, все скорости масштабируются с сохранением

траектории.

4.2. Одометрия (`rus_slam_base/odometry_core.py`)

Вход: фактические углы модулей и пройденный путь каждого колеса (тики энкодера → метры). Каждое колесо дает вектор скорости $w_i = v_i \cdot [\cos\theta_i, \sin\theta_i]$, приложенный в точке (x_i, y_i) . Скорость корпуса (v_x, v_y, w_z) находится как решение переопределенной системы (МНК, нормальные уравнения 3×3). Это устойчиво к шуму отдельных энкодеров и корректно для крабового хода и вращения на месте.

4.3. Безопасность (`rus_slam_safety/safety_node.py`)

Гейт `/cmd_vel` останавливает работа при любом из событий:

1. Активен внешний e-stop (`/e_stop = true`).
2. Нет телеметрии модулей дольше `comms_timeout` (обрыв USB).
3. Напряжение АКБ ниже `battery_critical_v` (аварийно) или ниже `battery_low_v` (ползущий режим, ограничение 0.2).
4. Модуль сообщил флаг неисправности.

Ограничение скорости умножается на коэффициент `/safety/speed_limit` (0.0–1.0), задаваемый менеджером миссии (знак «Стоп», красный свет, «лежащий полицейский»).

4.4. Менеджер миссии (`rus_slam_safety/mission_manager_node.py`)

Конечный автомат: BOOT → HOMING → READY → DELIVER → DONE / FAULT. На этапе DELIVER последовательно отправляет точки маршрута из конфига `mission.yaml` через действие `NavigateToPose`. Реакции на восприятие: красный светофор → лимит 0 до зеленого; знак «Стоп» → полная остановка на `stop_hold_sec`; «Искусственная неровность» → лимит 0.3 на `slowdown_hold_sec`.

5. Протокол обмена с модулями (кратко)

Полное описание — в [SERIAL_PROTOCOL.md](#).

- **Команда ПК → МК** (10 байт): `0xAA 0x55` | ID | угол×100 (int16 BE) | ШИМ -1000..+1000 (int16 BE) | флаги | CRC16 BE.
 - **Телеметрия МК → ПК** (16 байт, 20 Гц): `0xBB 0x44` | ID | статус | угол | дельта энкодера | ШИМ | напряжение | ток | CRC16 BE.
 - CRC16 Modbus (полином `0xA001`). Сторожевой таймер МК: 300 мс без пакетов — полное обесточивание модуля.
-

6. Режимы запуска

Лаунч	Команда	Что поднимает
Только железо + кинематика	<code>ros2 launch rus_slam_bringup robot.launch.py</code>	description, мост, кинематика, одометрия, лидар, камера, детекторы, safety
Робот + навигация	<code>ros2 launch rus_slam_bringup full.launch.py</code>	всё выше + SLAM или локализация по карте + Nav2 + миссия
Симуляция без модулей	<code>ros2 launch rus_slam_bringup robot.launch.py sim:=true</code>	мост в режиме эмуляции модулей
Только картографирование	<code>ros2 launch rus_slam_navigation slam.launch.py</code>	slam_toolbox online_async
Навигация по готовой карте	<code>ros2 launch rus_slam_navigation navigation.launch.py map:=/path/to/map.yaml</code>	Nav2 + AMCL

Типовой порядок ввода в эксплуатацию:

1. `sim:=true` — проверка графа топиков без железа;
2. прошивка 4 плат (firmware/module_controller), установка udev-правил (`rus_slam_bringup/udev/99-rus-slam.rules`);
3. хоминг: `ros2 service call /home_modules rus_slam_interfaces/srv/HomeModules {}`;
4. картирование: записать карту через `slam_toolbox + map_saver_cli`;
5. боевой запуск: `full.launch.py use_sim_time:=false map:=...`

7. Конфигурация

Все параметры вынесены в YAML-файлы пакета `rus_slam_bringup/config`:

Файл	Содержимое
<code>robot.yaml</code>	геометрия шасси, порты модулей, лимиты скоростей, параметры одометрии
<code>lidar.yaml</code>	порт/скорость лидара, параметры кадра протокола, <code>frame_id</code>
<code>perception.yaml</code>	устройство камеры, пороги детекторов, области интереса
<code>safety.yaml</code>	таймауты, пороги АКБ, статусы
<code>mission.yaml</code>	точки маршрута, время остановки у знаков
<code>nav2_params.yaml</code> (в <code>rus_slam_navigation/config</code>)	полный стек Nav2 под шасси 750×600 мм
<code>slam_toolbox.yaml</code>	online_async SLAM

8. Отказоустойчивость

Отказ	Реакция системы
Обрыв USB модуля	Сторож МК (300 мс) обесточивает модуль; safety_node останавливает робота по вотчдогу телеметрии
Потеря пакетов/битый CRC	Пакет отбрасывается, кадр синхронизируется по заголовку
Просадка АКБ	Двухпороговая политика: ползущий режим → аварийный стоп
Смерть ноды	Лаунчеры respawn:=true; миссия при FAULT останавливает робота через лимит 0
Нет признаков (лидар молчит)	lds01_driver_node публикует пустые сканы и статус, costmap считает лучи свободными только по свежим данным

9. Направления развития (не входит в текущую ревизию)

1. Замена классических CV-детекторов на YOLOv8-модель с весами, обученными на знаках полигона.
2. Фильтр локализации robot_localization (EKF), если появится IMU.
3. Behavior Tree (py_trees/BehaviorTree.CPP) вместо линейного FSM миссии.
4. Интерфейс оператора: веб-панель (телеметрия, телеуправление, карта).
5. Диагностика diagnostic_msgs по всем узлам.